COLLEGE OF COMPUTING AND INFORMATION SCIENCES

SCHOOL OF COMPUTING AND INFORMATICS TECHNOLOGY

DEPARTMENT OF NETWORKS

BSE 4202: SOFTWARE SECURITY

PRACTICAL TAKE HOME ASSIGNMENT

Assignment Title:	SecureApp: Build, Break, and Fix — A Full-Stack Security Assessment
Total Marks:	**40 Marks**
Weight:	30% of Final Grade
Date Issued:	28th April 2026
Submission Deadline:	13th May 2026, 11:59 PM EAT
Submission Mode:	MUELE (Moodle) — Upload ZIP archive + PDF report
Work Mode:	Group Assignment (plagiarism detection will be applied)
Lecturer:	Dr. Drake Patrick Mirembe, PhD, www.drakemirembe.org
Presentation date	15th May, 2026

Assignment Overview

This assignment requires you to apply software security principles to a realistic full-stack web application scenario. You are provided with a deliberately vulnerable web application called ***8TechBank*** a simulated online banking portal. Your task is to: (1) conduct a security audit identifying vulnerabilities; (2) demonstrate exploitation of specific attack vectors; (3) implement secure code fixes applying defense-in-depth principles; (4) design API security controls; and (5) produce a professional security assessment report aligned with industry frameworks.

The assignment is structured into five tasks of increasing complexity, each targeting specific course learning outcomes. All tasks are performed on the same application codebase, simulating the work of a professional security engineer conducting a comprehensive security review.

> ⚠ **ACADEMIC INTEGRITY WARNING**
>
> This is a group assignment. Each ground must submit their own original work and each must of the group is expected to clear indicate which areas of the assignment they made an individual contribution. Submissions will be checked using Turnitin and code similarity detection tools. Sharing code, copying from online sources without attribution, or submitting another student's work constitutes academic misconduct and will result in a zero mark and disciplinary referral. You may consult course materials, textbooks, and official documentation, but all code and analysis must be your own.

Learning Outcomes Assessed

1. LO1: Identify and classify software vulnerabilities using established taxonomies (CWE, OWASP Top 10).
2. LO2: Demonstrate practical exploitation of common web application vulnerabilities (XSS, SQL Injection, CSRF).
3. LO3: Apply defense-in-depth strategies to secure application code, APIs, and infrastructure.
4. LO4: Design and implement input validation, output encoding, authentication, and authorization controls.
5. LO5: Conduct a structured security audit and produce a professional security assessment report.
6. LO6: Apply cybersecurity frameworks (NIST, OWASP ASVS) to evaluate application security posture.

8TechBank Application Specification

You are required to build a minimal but functional version of 8TechBank using the following stack. The application must include the deliberately vulnerable code provided in each task, which you will then audit, exploit, and fix.

Component	Specification
Backend	Python (Flask) or Node.js (Express) your choice
Database	SQLite (for simplicity; SQL injection must be demonstrable)
Frontend	HTML/CSS/JavaScript (vanilla or minimal framework)
Authentication	Session-based login with username/password
Core Features	User registration, login, account dashboard, fund transfer between accounts, transaction history, admin panel
Deployment	Local development server (localhost). Docker optional for bonus marks.

TASK 1: Security Audit & Vulnerability Identification

Marks: 8/40 | Learning Outcomes: LO1, LO5, LO6

You are the newly hired application security engineer at 8TechBank. Your first assignment is to conduct a comprehensive security audit of the application's codebase. The CTO suspects the development team has shipped code with multiple security flaws and needs a professional assessment.

1.1 Instructions

1. Build the 8TechBank application with the following deliberately vulnerable code patterns embedded in the codebase (you must include all six):
2. Conduct a manual code review of your application and identify at least 8 distinct vulnerabilities.
3. For each vulnerability, provide: (a) the CWE identifier and OWASP Top 10 category; (b) the exact file name and line number(s) where the vulnerability exists; (c) a severity rating using CVSS v3.1 scoring; (d) a clear description of the security impact; and (e) a recommended remediation.
4. Produce a Vulnerability Assessment Matrix (table) summarizing all findings, sorted by severity (Critical → Low).

1.2 Vulnerable Code Patterns to Embed

Include the following six vulnerable patterns in your application code:

Pattern 1: SQL Injection in Login (Python/Flask example)

```
# VULNERABLE - String concatenation in SQL query
@app.route('/login', methods=['POST'])
def login():
```

```
    username = request.form['username']
    password = request.form['password']
    query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"
    user = db.execute(query).fetchone()
    if user:
        session['user_id'] = user['id']
        return redirect('/dashboard')
```

Pattern 2: Reflected XSS in Search

```
# VULNERABLE - User input rendered without encoding
@app.route('/search')
def search():
    query = request.args.get('q', '')
    return f"<h2>Results for: {query}</h2><p>No results found.</p>"
```

Pattern 3: Stored XSS in Transaction Notes

```
# VULNERABLE - User-supplied note stored and rendered raw
@app.route('/transfer', methods=['POST'])
def transfer():
    note = request.form['note']  # No sanitization
    db.execute("INSERT INTO transactions (from_acct, to_acct, amount, note) VALUES (?,?,?,?)",
               (session['user_id'], to_acct, amount, note))
# In template: {{ note | safe }}  <-- disables auto-escaping
```

Pattern 4: Broken Access Control (IDOR)

```
# VULNERABLE - No authorization check on account access
@app.route('/account/<int:account_id>')
def view_account(account_id):
    account = db.execute("SELECT * FROM accounts WHERE id=?", (account_id,)).fetchone()
    return render_template('account.html', account=account)
    # Any authenticated user can view ANY account by changing the ID
```

Pattern 5: Plaintext Password Storage

```
# VULNERABLE - Passwords stored in plaintext
@app.route('/register', methods=['POST'])
def register():
    username = request.form['username']
    password = request.form['password']  # No hashing!
    db.execute("INSERT INTO users (username, password) VALUES (?,?)", (username, password))
```

Pattern 6: Missing CSRF Protection on Fund Transfer

```
# VULNERABLE - No CSRF token validation on state-changing action
@app.route('/transfer', methods=['POST'])
def transfer():
    to_account = request.form['to_account']
    amount = request.form['amount']
    # Proceeds without verifying CSRF token
    execute_transfer(session['user_id'], to_account, amount)
```

TASK 2: Exploit Demonstration — Attack Proof of Concept

Marks: 10/40 | Learning Outcomes: LO2, LO5

Now that you have identified the vulnerabilities, demonstrate that they are real and exploitable. For each of the following four attack vectors, write and execute a working proof-of-concept (PoC) exploit against your 8TechBank application. Document each exploit with screenshots showing successful exploitation.

2.1 Required Exploit Demonstrations

Exploit A: SQL Injection Authentication Bypass (3 marks)

Demonstrate bypassing the login form using SQL injection. Show: (a) the exact payload you inject into the username and/or password field; (b) a screenshot of successful login as another user without knowing their password; (c) a second payload that extracts all usernames and passwords from the database (UNION-based or blind SQLi).

Exploit B: Cross-Site Scripting (XSS) Session Hijacking (3 marks)

Demonstrate both reflected and stored XSS attacks. Show: (a) a reflected XSS payload in the search field that triggers a JavaScript alert displaying the session cookie (document.cookie); (b) a stored XSS payload in the transaction notes field that executes whenever any user views transaction history; (c) explain how an attacker would use the stolen session cookie to hijack a user's session (you do not need to demonstrate a full hijack, but provide the step-by-step attack narrative).

Exploit C: CSRF Unauthorized Fund Transfer (2 marks)

Create a malicious HTML page that, when opened by an authenticated 8TechBank user in the same browser, automatically submits a fund transfer request to the attacker's account without the victim's knowledge. Provide: (a) the complete malicious HTML code; (b) a screenshot showing the transfer was executed successfully; (c) an explanation of why the application was vulnerable to this attack.

Exploit D: IDOR Unauthorized Account Access (2 marks)

Demonstrate accessing another user's account details by manipulating the account ID in the URL. Show: (a) login as User A (e.g., account ID 1); (b) modify the URL to access User B's account (e.g., account ID 2); (c) screenshot showing you can view User B's balance, transactions, and personal information.

> **ETHICAL USE NOTICE**
>
> These exploit techniques must ONLY be used against your own 8TechBank application running on localhost. Attempting to use these techniques against any real system, university infrastructure, or another student's application constitutes a criminal offence under Uganda's Computer Misuse Act, 2011 and will result in immediate disciplinary action. The Lecturer is not responsible for your criminal or illegal actions.

TASK 3: Secure Code Implementation Defense in Depth

Marks: 10/40 | Learning Outcomes: LO3, LO4

Apply defense-in-depth principles to fix every vulnerability you identified and exploited. For each fix, provide the vulnerable code (before) and the secure code (after) with inline comments explaining the security rationale. Your fixes must address the root cause, not just apply superficial patches.

3.1 Required Fixes (implement all six)

Fix 1: Parameterized Queries (2 marks)

Replace all string-concatenated SQL queries with parameterized queries (prepared statements). Show before/after code. Demonstrate that the SQL injection payload from Task 2 no longer works against the fixed code (screenshot required).

Fix 2: Output Encoding & Content Security Policy (2 marks)

Implement proper output encoding for all user-generated content rendered in HTML. Implement a Content Security Policy (CSP) header that blocks inline script execution. Show before/after code and demonstrate that the XSS payloads from Task 2 are neutralized.

Fix 3: CSRF Token Implementation (1.5 marks)

Implement CSRF token generation and validation on all state-changing forms (registration, login, fund transfer). Show the token generation mechanism, form embedding, and server-side validation code. Demonstrate the CSRF attack page from Task 2 now fails.

Fix 4: Authorization & Access Control (1.5 marks)

Implement proper authorization checks ensuring users can only access their own accounts. Show the middleware/decorator pattern used and demonstrate the IDOR exploit from Task 2 now returns a 403 Forbidden response.

Fix 5: Password Hashing with bcrypt (1.5 marks)

Replace plaintext password storage with bcrypt hashing (minimum 12 rounds). Show the registration code storing hashed passwords and the login code verifying against hashes. Include a screenshot of the database showing hashed (not plaintext) passwords.

Fix 6: Security Headers & Session Hardening (1.5 marks)

Implement the following security headers: X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Strict-Transport-Security, and Referrer-Policy: no-referrer. Implement secure session configuration: HttpOnly, Secure, SameSite=Strict cookie flags, and session timeout (15 minutes). Show the configuration code.

TASK 4: API Security & Sandboxing

Marks: 6/40 | Learning Outcomes: LO3, LO4

8TechBank needs a secure REST API for its planned mobile application. Design and implement a secure API layer with the following requirements:

4.1 API Authentication & Authorization (2 marks)

Implement JWT-based API authentication. Include: (a) a /api/auth/token endpoint that issues JWTs upon valid credential submission; (b) JWT validation middleware that protects all API routes; (c) role-based access control distinguishing 'user' and 'admin' roles; (d) token expiry (15 minutes) and refresh mechanism. Provide the complete authentication flow code with security-relevant comments.

4.2 API Rate Limiting & Input Validation (2 marks)

Implement: (a) rate limiting on the authentication endpoint (maximum 5 attempts per minute per IP) to prevent brute-force attacks; (b) input validation on all API endpoints using a schema validation library (e.g., Marshmallow, Joi, Pydantic); (c) demonstrate that malformed or oversized inputs are rejected with appropriate HTTP error codes (400, 422). Provide test cases showing the rate limiter and validator in action.

4.3 Application Sandboxing Design (2 marks)

Write a 500-word design document describing how you would sandbox 8TechBank in a production environment. Address: (a) containerization using Docker with a least-privilege user; (b) network segmentation separating the web server, application server, and database; (c) file system restrictions (read-only root filesystem, writable only in /tmp); (d) resource limits (CPU, memory, process count); (e) provide a Dockerfile and docker-compose.yml implementing your design (these can be descriptive/pseudocode if Docker is not installed, but must be syntactically correct).

TASK 5: Security Assessment Report

Marks: 6/40 | Learning Outcomes: LO5, LO6

Produce a professional security assessment report (PDF, 8–12 pages) that consolidates your work from Tasks 1–4 into a single deliverable suitable for presentation to 8TechBank's executive leadership. The presentation will be done on 15th May, 2026. The report must follow the structure below:

Executive Summary (1 page, 1 mark)

A non-technical summary of findings, overall risk assessment (Critical/High/Medium/Low), and 3 prioritized recommendations. Written for C-suite audience.

Methodology (0.5 pages, 0.5 marks)

Describe the audit approach: manual code review, dynamic testing, tools used, and scope boundaries. Reference OWASP Testing Guide v4.2 methodology.

Findings & Risk Analysis (3–4 pages, 2 marks)

Present each vulnerability with: description, reproduction steps, evidence (screenshots), CVSS score, business impact, and remediation status (fixed/pending). Use a traffic-light risk rating system.

Remediation Summary (1–2 pages, 1 mark)

For each vulnerability, present the before/after code comparison and verification that the fix works (screenshot of failed exploit attempt post-fix).

OWASP ASVS Compliance Assessment (1 page, 1 mark)

Map your application against the OWASP Application Security Verification Standard (ASVS) Level 1 requirements. Use a compliance matrix showing Pass/Fail/Partial for at least 10 relevant ASVS controls. Identify gaps and recommend actions to achieve full Level 1 compliance.

Appendices (0.5 marks)

Include: (A) complete vulnerability assessment matrix; (B) tool output logs; (C) code snippets not included in the main report.

Submission Requirements

1. Upload a single ZIP archive to MUELE named: StudentID_CSC4207_Assignment.zip
2. The ZIP must contain: (a) /src/ — complete source code with both vulnerable and fixed versions in separate directories (/src/vulnerable/ and /src/secure/); (b) /exploits/ — all exploit scripts and PoC files; (c) /screenshots/ — all evidence screenshots referenced in the report; (d) /report/ — the security assessment report as PDF; (e) README.md with setup instructions to run the application.
3. The application must run successfully from the submitted code. Include a requirements.txt or package.json and clear setup instructions. The marker will attempt to run your application; non-functional submissions will lose marks across all tasks.
4. Late submissions: 5 marks deducted per day late, up to 3 days. Submissions after 3 days late will not be accepted.
5. AI tool disclosure: If you used any AI coding assistants (GitHub Copilot, ChatGPT, etc.) for any part of this assignment, you must disclose this in a section of your report titled 'AI Tool Usage Declaration' specifying which tools were used, for which tasks, and how the output was modified. Undisclosed AI use detected by similarity tools will be treated as academic misconduct.

Recommended Resources

1. OWASP Top 10 (2021): https://owasp.org/Top10/
2. OWASP Testing Guide v4.2: https://owasp.org/www-project-web-security-testing-guide/
3. OWASP Application Security Verification Standard (ASVS) v4.0.3
4. MITRE CWE Database: https://cwe.mitre.org/
5. NIST CVSS v3.1 Calculator: https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator
6. Howard & LeBlanc (2009). 24 Deadly Sins of Software Security. McGraw-Hill.
7. Stuttard & Pinto (2011). The Web Application Hacker's Handbook, 2nd ed. Wiley.
8. Flask Security Best Practices: https://flask.palletsprojects.com/en/latest/security/

FINAL NOTE FROM THE LECTURER

This assignment is designed to simulate real-world application security work. Approach it as a professional engagement: thoroughness, accuracy, and clear communication matter as much as technical skill. The best submissions will demonstrate not just the ability to find and fix vulnerabilities, but the judgment to prioritize risks and communicate them effectively to non-technical stakeholders. Good luck.